UniCaronas

Sistema de Caronas Universitárias

Relatório de Resultados — Sprint 3

Disciplina: Prática em Programação Web — 3ESCN

Equipe: Rafael Machado, Ariane Archanjo, Pedro Kafka, Matheus Sizanoski

---

1. Resumo Executivo

A Sprint 3 do projeto UniCaronas foi concluída com sucesso no período de 09 a 15 de abril de 2026. Todas as histórias planejadas foram entregues dentro do prazo, totalizando 13 Story Points. O sistema passou a contar com chat em tempo real entre motorista e passageiro, fluxo de pagamento simulado com cálculo de taxa, e dois itens adicionais entregues além do escopo original: visualização de mapa interativo na tela de detalhes da carona e melhorias gerais de responsividade em toda a interface.

Métrica	Valor
Story Points planejados	13 SP
Story Points entregues	13 SP (100%)
Histórias concluídas	3 de 3 (US08, US08b, US09)
Entregas adicionais	Mapa interativo (Leaflet.js) + melhorias de responsividade
Velocidade	13 SP / semana
Impedimentos resolvidos	1 de 1
Status geral	✅Sprint concluída

---

2. Histórias de Usuário Entregues

US08 — Chat entre usuários (5 pontos)

Como motorista ou passageiro, quero me comunicar via chat com o outro usuário após a solicitação, para combinar detalhes como ponto de encontro e horário exato.

Critério de Aceitação	Status
Chat disponível após solicitação aceita pelo motorista	✅Implementado
Histórico de mensagens carregado ao abrir o chat	✅Implementado
Mensagens novas aparecem sem recarregar a página (polling 5s)	✅Implementado
Indicação visual de mensagens não lidas (badge na navbar)	✅Implementado
Apenas motorista e passageiro da carona acessam o chat	✅Implementado

US08b — Aceitar/Recusar solicitação pela interface (3 pontos)

Como motorista, quero visualizar solicitações pendentes e aceitar ou recusar cada uma diretamente na tela, sem precisar usar ferramentas externas.

Critério de Aceitação	Status
Motorista vê lista de solicitações pendentes na tela da carona	✅Implementado
Botões Aceitar e Recusar funcionam e atualizam o status	✅Implementado
Vaga é descontada ao aceitar; devolvida ao recusar (trigger)	✅Implementado
Passageiros não veem a seção de gerenciamento de solicitações	✅Implementado

US09 — Realizar pagamento (5 pontos)

Como passageiro com vaga confirmada, quero realizar o pagamento dentro da plataforma para que a transação fique registrada.

Critério de Aceitação	Status
Tela exibe detalhamento de valores (total, taxa 10%, repasse motorista)	✅Implementado
Passageiro seleciona método (cartão, PIX, dinheiro) e confirma	✅Implementado
Sistema retém 10% como taxa da plataforma no registro	✅Implementado
Duplo pagamento bloqueado com erro 409 e mensagem clara	✅Implementado
Confirmação visual após pagamento bem-sucedido	✅Implementado

---

3. Entregas Adicionais

Mapa Interativo — Leaflet.js

A tela de detalhes da carona (carona.html) passou a exibir um mapa interativo mostrando o trajeto entre origem e destino. A implementação utiliza a biblioteca Leaflet.js (v1.9.4) combinada com a API pública Nominatim (OpenStreetMap) para geocodificação dos endereços de texto.

Como funciona:

- Ao carregar os detalhes de uma carona, os campos origem e destino são enviados à API Nominatim para obter coordenadas geográficas
- Dois marcadores são posicionados no mapa — um para a origem e outro para o destino
- O mapa é ajustado automaticamente para enquadrar os dois pontos (fitBounds com padding)
- Se o endereço não for reconhecido pela API, o mapa é ocultado silenciosamente sem erros visíveis
- Estilização integrada ao design escuro do sistema — borda, border-radius e z-index consistentes

Melhorias de Responsividade

A interface passou por uma revisão geral de responsividade, garantindo que todas as telas funcionem bem em dispositivos móveis e tablets. As principais melhorias foram:

- Navbar com menu hamburguer em telas menores — links ocultos colapsam em menu vertical ao clicar
- Stats grid reajustado de 3 para 2 colunas em telas até 768px e 1 coluna abaixo de 480px
- Filtros de busca empilhados verticalmente em mobile — botão Buscar com largura total
- Carona card com footer reorganizado em coluna em telas pequenas
- Tela de detalhes da carona com layout de coluna única abaixo de 992px — sidebar desce para baixo do conteúdo principal
- Cadastro com grid de 2 colunas revertendo para 1 coluna em mobile
- Avaliações com rota exibida em bloco separado abaixo do header em telas pequenas

---

4. Telas Implementadas / Atualizadas

Tela de Detalhes da Carona — carona.html (atualizada)

Layout em duas colunas com mapa interativo no topo da coluna principal. Seção de solicitações pendentes visível apenas para o motorista com botões de aceitar e recusar inline. Sidebar com preço, botão de solicitação (passageiro) ou conclusão (motorista), e botão de chat aparecendo após aceite. Em mobile, a sidebar sobe para logo abaixo do mapa.

Tela de Pagamento — pagamento.html (nova)

Tela centralizada com card de checkout. Exibe resumo com três linhas: valor da carona, taxa da plataforma (deduzida, em vermelho) e total a pagar. Linha adicional mostra o repasse ao motorista. Seleção de método de pagamento em três botões visuais (cartão, PIX, dinheiro) com estado ativo destacado. Botão de confirmação desabilita e muda texto durante processamento.

Chat Global — chat-global.js (novo módulo)

Widget de chat flutuante injetado via script em todas as páginas autenticadas. Botão FAB no canto inferior direito com badge de mensagens não lidas. Painel deslizante com lista de conversas e tela de mensagens com navegação entre elas. Polling automático a cada 5 segundos. Mensagens do usuário alinhadas à direita (bolha roxa) e do outro à esquerda (bolha escura). Persistência de estado aberto/fechado via localStorage.

---

5. Principais Componentes de Código

5.1 Backend — Endpoints utilizados nesta Sprint

Arquivo	Responsabilidade
controllers/mensagensController.js	Envio, listagem e contagem de não lidas — acesso restrito aos participantes da carona

Arquivo	Responsabilidade
controllers/pagamentosController.js	Processamento simulado de pagamento com cálculo de taxa (10%) e repasse — referência externa gerada localmente
routes/mensagens.js	POST /api/mensagens, GET /api/mensagens/:id, GET /api/mensagens/nao-lidas
routes/pagamentos.js	POST /api/pagamentos, GET /api/pagamentos/historico

5.2 Frontend — Módulos novos

Arquivo	Descrição
js/chat-global.js	Widget de chat auto-inicializado: injeção de HTML/CSS, polling, lista de conversas, envio de mensagens — expõe UCChat.iniciarChat() para qualquer página
pages/pagamento.html	Tela de checkout com seleção de método, resumo de valores e integração com POST /api/pagamentos
Leaflet.js (CDN)	Biblioteca de mapas integrada via CDN (v1.9.4) com tiles do OpenStreetMap e geocodificação via Nominatim

5.3 Fluxo completo Sprint 3

Após aceite da solicitação: passageiro vê badge de vaga confirmada → link para pagamento → abre tela com resumo de valores → seleciona método → confirma → registro salvo no banco com taxa calculada → botão de chat disponível para ambos → troca de mensagens com histórico e polling.

---

6. Impedimentos e Resoluções

Impedimento	Impacto	Resolução
Nominatim bloqueando requisições em sequência rápida (rate limit)	Mapa não carregava em alguns casos	Adicionado try/catch silencioso — mapa oculto quando geocodificação falha, sem erro na tela

---

7. Retrospectiva da Sprint

O que funcionou bem

- Chat com polling funcionou sem necessidade de WebSocket — solução simples e eficaz para o contexto acadêmico
- Módulo chat-global.js desacoplado permitiu reutilização em todas as páginas sem duplicação de código
- Mapa interativo elevou significativamente a percepção de qualidade do produto
- Melhorias de responsividade resolveram problemas visíveis em apresentações com dispositivos móveis
- Sprint de 1 semana com 100% de entrega — planejamento bem calibrado ao tempo disponível

O que pode melhorar

- Polling de 5s gera muitas requisições — avaliar WebSocket ou Server-Sent Events em versão futura
- Geocodificação por texto livre é imprecisa para endereços curtos — campo de busca por coordenadas seria mais robusto
- Integração de pagamento real (Stripe / PagSeguro) ainda pendente — simulação suficiente para o escopo acadêmico

---

8. Tecnologias Utilizadas

Camada	Tecnologia	Versão / Detalhe
Frontend	HTML5 + CSS3 + JavaScript	Vanilla JS — sem frameworks
Mapas	Leaflet.js + Nominatim	v1.9.4 — tiles OpenStreetMap, geocodificação pública
Backend	Node.js + Express	Express 4.x
Banco de Dados	PostgreSQL	v14+ — tabelas mensagens_chat e pagamentos
Autenticação	JWT (jsonwebtoken)	v9.0 — todas as rotas de chat e pagamento protegidas
Chat (real-time)	Polling via setInterval	5s — implementado no chat-global.js
Versionamento	GitHub	Branch principal — código estável e testado